

Performance Testing

Date	01 October 2026
Team ID	6ab7f752d083b0e2ee72b0bf
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how PocketSmart AI behaves under expected and peak load.

Step 1: Testing Overview

Testing Tool Used	Postman (API request testing of the login, planner and health endpoints) and a Python load script (requests + threads, psutil) for concurrent users, CPU and memory
Type of Testing	Load Testing and Stress Testing
Target Module / API	POST /planner/home, /planner/party and /planner/jewelry (with image upload); GET /, /login, /register and /dashboard
Test Environment	Local machine (1 CPU core(s), 4 GB RAM), Flask server on 127.0.0.1 with SQLite storage. The Gemini call was replaced by a mock that returns a fixed plan after a 1.5 second delay, so results show application behaviour excluding real Gemini latency.
Test Date	01-10-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline: single user generates a Home Interior plan	1	30	Valid plan returned, no errors
2	Load: concurrent users submit Home Planner requests	10	60	Average response within target; error rate under 1%
3	Load: concurrent users submit Party Planner requests	10	60	Plan returned for all users
4	Stress: Jewelry Planner requests with outfit image upload	25	60	App stays up and returns plans; no crash
5	Page loads: landing, login, register and dashboard pages	10	30	Pages load in under 2 seconds

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.51 s (Home, 10 users); 1.52 s (Jewelry, 25 users)	Pass	Includes the 1.5 s mocked Gemini delay. Page loads averaged 21 ms.
2	Response Time (Max)	< 5 seconds	1.62 s	Pass	Slowest request across all scenarios.
3	Throughput (Req/sec)	-	0.66 (1 user) to 471.90 (25 users)	-	Page-load test reached 471.9 req/s.

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
4	Error Rate	< 1%	0.00%	Pass	Requests returning anything other than a valid page.
5	CPU Utilization	< 80%	Peak 56.0% (server process, share of all cores)	Pass	Average 50.6% in the busiest scenario.
6	Memory Utilization	< 80%	Peak 1.13% of RAM (about 47 MB)	Pass	Server process resident memory.

Step 4: Observations & Analysis

- The Home planner served 10 concurrent users at 6.60 requests per second with an average of 1.51 s and no errors; the Party planner behaved the same (6.60 req/s, 1.51 s).
- The stress scenario with 25 users uploading an image reached 16.40 req/s with an average of 1.52 s and a maximum of 1.62 s; the image upload added no visible delay and no request failed.
- Slowest endpoint: the planner POST routes, because each request waits for the AI call (1.5 s in this test). With the real Gemini API the response time will follow Gemini latency and quota, so the 10-second target for AI plans must be re-checked with a live key.
- Page loads were fast (21 ms average, 97 ms maximum) and CPU and memory stayed far below the 80% limits, so the application layer is not the bottleneck.
- Fallback behaviour was not triggered: the application shows an error message when Gemini fails instead of substituting default recommendations.

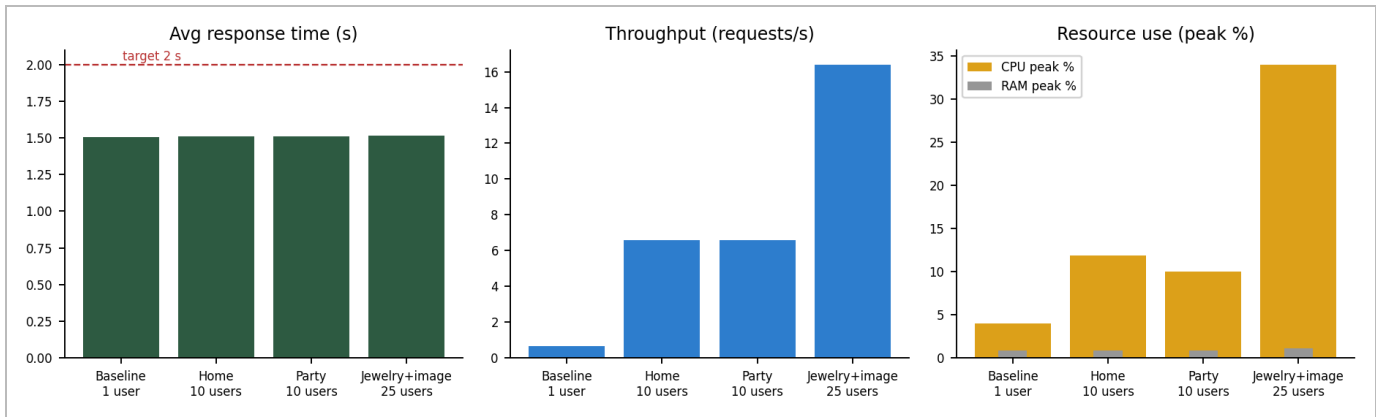
Postman API Test Cases

The team exercised the endpoints below with Postman. Because the application uses Flask-Login sessions, the planner requests are sent after a successful login so that Postman reuses the session cookie.

S.No	Request	Input (Postman body / form-data)	Expected response
1	GET /api/health	None	200 OK with {"status": "ok", "service": "PocketSmart AI"}
2	POST /register	name, email, password (8+ characters)	Redirect to /login with an account-created message; a duplicate email redirects to /login with a warning
3	POST /login	email, password	Redirect to /dashboard; wrong password stays on /login with "Invalid email or password."
4	POST /planner/home	budget, rooms, style, requirements	200 OK page with budget strip, allocations, recommendation cards and tips
5	POST /planner/party	budget, event_type, guests, city, preferences	200 OK page with the party plan; guests below 1 shows an error message
6	POST /planner/jewelry	budget, occasion, style, metal, preferences, image (file)	200 OK page with jewelry options; a non-image file shows "Only PNG, JPG, JPEG and WEBP images are allowed."
7	GET /history/<id>	id of a saved plan	200 OK with the saved plan; another user's id returns 404
8	GET /dashboard (not logged in)	None	Redirect to /login

Step 5: Screenshots / Evidence

Chart of the measured load-test results (average response time, throughput and peak resource use per scenario):



Load-test results chart

Measured values recorded by the test script for each scenario:

Scenario	Users	Requests	Avg (s)	95th pct (s)	Max (s)	Req/s	Errors (%)	CPU avg / max (%)	RAM max (%)
baseline home	1	20	1.51	1.51	1.52	0.66	0.0	0.3 / 4.0	0.82
load home	10	400	1.51	1.52	1.54	6.60	0.0	2.2 / 11.9	0.88
load party	10	400	1.51	1.52	1.55	6.60	0.0	2.1 / 10.0	0.88
stress jewelry	25	1000	1.52	1.54	1.62	16.40	0.0	9.8 / 34.0	1.13
pages	10	14228	0.02	0.03	0.10	471.90	0.0	50.6 / 56.0	1.12